

零基础到底层原理教程

从第一条 SQL、表设计、导入，到 MPP / CBO / Pipeline / Compaction 与生产排障

学习路线	你最终能做到
第 1 层 · 会用	独立部署 3.0.8；建表、导入、查询、更新、备份思维
第 2 层 · 会设计	按业务选择表模型、排序键、分区、分桶、索引
第 3 层 · 懂原理	解释 FE/BE、Tablet/Replica、Rowset/Segment、MPP、CBO、Pipeline
第 4 层 · 会排障	用日志、SHOW、EXPLAIN、Profile 建立证据链；读 GitHub issue 不盲抄答案

版本边界

本文固定学习 Apache Doris 3.0.8。3.0.8 于 2025-09-19 发布；当前官网的 3.x 页面可能持续修订，因此凡可能越过 3.0.8 的内容均以“需核对版本”标注。

阅读约定

【概念】给零基础释义；【底层】解释实现原因；【实验】要求亲手验证；【警告】指出生产风险。

目录

建议严格按顺序完成第 0-7 章；第 8-12 章可作为进阶与排障手册。

零基础到底层原理教程	1
目录	2
使用方法与 6 周学习计划	5
第 0 章：从零补齐数据库语言	6
0.1 数据库、表、行、列到底是什么	6
0.2 先学会读一条 SQL	6
0.3 必备计算机概念注解	6
0.4 自测	6
第 1 章：认识 Doris 3.0.8	7
1.1 一句话定位	7
1.2 FE 与 BE	7
1.3 两种部署架构	7
1.4 3.0.8 要特别知道什么	7
第 2 章：部署第一个 3.0.8 集群	8
2.1 学习环境选择	8
2.2 Docker 快速启动	8
2.3 手工二进制部署的最小闭环	8
2.4 端口速查与连通性	8
2.5 第一个健康检查脚本思路	9
2.6 实验	9
第 3 章：建表是最重要的性能设计	9
3.1 四个不可逃避的问题	9
3.2 三种表模型	9
3.3 三张教学表	10
3.4 用数据证明模型不同	10
第 4 章：一行数据怎样落到 Tablet	11
4.1 分区与分桶是两层切分	11
4.2 Tablet 数量估算	11
4.3 分桶键选择	11
4.4 副本与多数派	11
4.5 实验	11
第 5 章：导入、事务与数据可见性	12
5.1 如何选择导入方式	12

5.2 Stream Load 完整实验	12
5.3 写入内部发生了什么	12
5.4 脏数据治理	12
第 6 章：列式存储、Rowset、Segment 与 Compaction	13
6.1 为什么列式适合分析	13
6.2 存储层级	13
6.3 Compaction 为什么必需	13
6.4 Unique Key 的 Merge-on-Write 直觉	14
6.5 实验与观察	14
第 7 章：SQL 如何变成分布式计算	14
7.1 从文本到结果的流水线	14
7.2 MPP 与 Fragment	14
7.3 Join 分发策略	14
7.4 Pipeline 为什么替代 Volcano	15
7.5 向量化与 SIMD	15
7.6 Runtime Filter	15
第 8 章：EXPLAIN、Profile 与系统化调优	15
8.1 不要从调参数开始	15
8.2 EXPLAIN 看什么	15
8.3 Profile 看什么	15
8.4 常见反模式	16
第 9 章：生产运维与故障思维	17
9.1 四个黄金信号	17
9.2 日志与证据链	17
9.3 高可用基础	17
9.4 升级 3.0.8 前后清单	17
9.5 安全最小线	17
第 10 章：从 GitHub 真实问题学习排障	18
案例 A：3.0.8 Docker FE 在 cgroup v2 启动失败 (#56784)	18
案例 B：删除 BE storage 后容器等待 FE Master (#58808)	18
案例 C：FROM_UNIXTIME 边界与文档理解 (#57576)	18
案例 D：JDBC Catalog 与 query() TVF 字段表现不一致 (#60163)	18
案例 E：把 3.0.8 Release 修复当作排障索引	19
第 11 章：源码地图 - 从概念找到实现	20
11.1 用一次慢查询读源码	20
11.2 一条 SQL 的源码追踪练习	20
第 12 章：综合项目 - 实时订单分析仓库	21

12.1 需求 21

12.2 设计答案 21

12.3 验收实验 21

12.4 你必须能口述的答案 21

附录 A：基础概念注解表 22

附录 B：面试与自测题（含简答） 23

 B.1 基础题 23

 B.2 原理题 23

 B.3 排障题 23

附录 C：资料、版本与引用 24

使用方法与 6 周学习计划

这不是“把命令复制一遍”的速查表。每章按 问题 → 操作 → 现象 → 原理 → 自测组织。第一次阅读先完成主线实验；第二次阅读再进入底层框。

周	目标	必须交付
1	计算机、Linux、SQL、OLAP 基础；部署单机 3.0.8	能解释端口、进程、文件、客户端；保留一份启动日志
2	三种表模型与完整建表	完成 logs/orders/daily_sales 三张表及对照实验
3	分区、分桶、Tablet、Replica；导入	完成 Stream Load；能说明一行数据落到哪里
4	列式存储、Rowset/Segment、Compaction、事务	画出写入到可见的状态图
5	CBO、MPP、Join、Pipeline、Runtime Filter	能读 EXPLAIN；能从 Profile 找慢算子
6	运维、故障演练、GitHub 案例、综合项目	提交一份排障报告和一套建模说明
完成标准 不看教程也能：部署 → 建表 → 导入 → 验数 → 优化 → 解释原理 → 对常见故障形成证据链。背下参数不算掌握。		

第 0 章：从零补齐数据库语言

0.1 数据库、表、行、列到底是什么

数据库是有组织的数据集合；表像带规则的二维表格；行是一条事实；列是事实的一个属性；Schema是字段、类型、约束的定义。SQL 是“声明我想要什么”的语言，数据库负责决定怎样执行。

【概念】OLTP 与 OLAP

OLTP服务下单、转账等高并发小事务，常按主键读写少量行；OLAP服务报表与分析，常扫描大量行、聚合少量列。Doris的主战场是实时 OLAP，不是替代所有交易数据库。

0.2 先学会读一条 SQL

```
SELECT city, SUM(amount) AS revenue
FROM orders
WHERE order_time >= '2026-08-01'
GROUP BY city
HAVING SUM(amount) > 1000
ORDER BY revenue DESC
LIMIT 10;
```

逻辑理解顺序：FROM 找数据 → WHERE 过滤行 → GROUP BY 分组 → 聚合 → HAVING 过滤组 → SELECT 输出 → ORDER BY 排序 → LIMIT 截断。优化器可能改写物理执行顺序，但必须保持 SQL 语义。

0.3 必备计算机概念注解

术语	零基础注解	与 Doris 的关系
CPU / 核	执行计算指令；多核可并行	BE 的向量化和 Pipeline 利用多核
内存	比磁盘快、断电丢失的工作区	Hash Join、排序、聚合大量使用；不足会失败或落盘
磁盘 / SSD	持久化数据，吞吐与延迟不同	存放 Segment、索引、日志、FE 元数据
网络	节点间传输数据	Shuffle/Exchange 可能成为查询瓶颈
进程 / 线程	运行中的程序 / 其执行单元	FE/BE 是进程；BE 内多任务并行
端口	一台机器上区分网络服务的号码	9030 MySQL 查询；8030 FE HTTP；8040 BE HTTP 等

0.4 自测

- 为什么分析查询更偏爱列式存储？
- WHERE 与 HAVING 分别过滤什么？
- “SQL 是声明式语言”对优化器意味着什么？

第 1 章：认识 Doris 3.0.8

1.1 一句话定位

Apache Doris 是兼容 MySQL 协议、面向实时分析的 MPP 数据库。它把“控制与规划”主要交给 FE，把“存储与执行”交给 BE，并通过列式存储、向量化、CBO、分布式并行和物化视图缩短分析延迟。

1.2 FE 与 BE

组件	职责	不要误解
FE (Frontend)	接入 SQL、鉴权、解析、优化、生成计划；管理表、分片、副本、节点等元数据	FE 通常不承担大规模数据扫描
BE (Backend)	存储数据；执行扫描、Join、聚合、排序等算子；参与导入与 Compaction	BE 不自行决定全局 SQL 语义
Broker / 外部组件	某些历史/特定导入路径使用；并非核心必需进程	不要把 Doris 理解为需要 Hadoop 全家桶

【底层】控制面与数据面
把“想做什么、在哪里做”与“真正读写计算”分开，能让 FE 统一管理元数据和计划，让多个 BE 水平扩展执行。但 FE 元数据与 BE 数据两条链都必须健康。

1.3 两种部署架构

模式	数据位置	适用思路
存算一体	BE 本地盘；Tablet 多副本分布在 BE	学习和多数自建场景更直观；本教程主线
存算分离（3.0 引入）	共享存储保存数据，BE 偏无状态并使用本地缓存；另有 Meta Service	需要独立弹性、工作负载隔离、对象存储成本优势时评估

1.4 3.0.8 要特别知道什么

- 3.0 起 Pipeline 模型完全替代旧 Volcano 执行模型。
- 3.0.8 改进点查询：优化器可规划为单 Fragment，Unique Key 点查也有性能改进。
- 3.0.8 默认关闭 VARIANT 的 nested flatten；需要时先设置 enable_variant_flatten_nested=true。
- 维护版修复覆盖导入、优化器、执行、复杂类型、Schema Change 和存算分离；升级前必须读行为变化。

【警告】版本选择
本教程为学习 3.0.8 而固定版本。生产环境是否继续使用 3.0.8，应结合当前受支持版本、安全修复、兼容性测试和升级策略单独决策。

第 2 章：部署第一个 3.0.8 集群

2.1 学习环境选择

最快路径是 Linux + Docker。macOS/Windows 可通过 Docker Desktop 学习，但生产部署应使用受支持的 Linux 环境。官方 Quick Start 明确提示：快速部署是开发测试用途，示例通常是单副本，容器销毁可能丢数据。

2.2 Docker 快速启动

```
# 从官方 Quick Start 页面下载 start-doris.sh 后
chmod 755 start-doris.sh
bash start-doris.sh -v 3.0.8

# 使用 MySQL 客户端验证
mysql -uroot -P9030 -h127.0.0.1 \
-e 'SELECT `host`,`join`,`alive` FROM frontends()'
mysql -uroot -P9030 -h127.0.0.1 \
-e 'SELECT `host`,`alive` FROM backends()'
```

【概念】为何用 MySQL 客户端

Doris 兼容 MySQL 通信协议和大量 SQL 语法，因此 mysql CLI、JDBC 和不少 BI 工具可连接它。但“协议兼容”不等于存储引擎或全部语义与 MySQL 相同。

2.3 手工二进制部署的最小闭环

1. 准备 Java 17；创建非 root 用户；配置足够的文件描述符和 vm.max_map_count。
2. fe.conf 设置 JAVA_HOME 与 priority_networks；启动 FE。
3. be.conf 设置 priority_networks 与 storage_root_path；启动 BE。
4. 连接 FE 的 9030 端口，执行 ALTER SYSTEM ADD BACKEND "be-host:9050"。
5. 用 SHOW FRONTENDS、SHOW BACKENDS 验证 Alive；再检查 FE/BE 日志。

2.4 端口速查与连通性

默认端口	组件	用途 / 检查方向
8030	FE	HTTP 管理与导入重定向相关
9010	FE	FE 节点间元数据通信
9020	FE	Thrift RPC
9030	FE	MySQL 协议 SQL 入口
8040	BE	BE HTTP
8060	BE	BE RPC
9050	BE	心跳服务；ADD BACKEND 使用
9060	BE	BE Thrift 服务

【排障顺序】连不上不是先重装

先确认进程 → 监听端口 → 本机连接 → 跨机路由/防火墙 → priority_networks → 主机名解析 → FE/BE 日志。每一步记录证据。

2.5 第一个健康检查脚本思路

```
SELECT VERSION();
SHOW FRONTENDS;
SHOW BACKENDS;
SHOW PROC '/frontends';
SHOW PROC '/backends';
```

2.6 实验

- 停止 BE 后观察 SHOW BACKENDS 的 Alive 和 LastHeartbeat；再启动并观察恢复。
- 用错误端口连接，解释 “connection refused” 和 “timeout” 的差异。
- 找到 fe.log、fe.warn.log、be.INFO / be.WARNING（文件名随包而异），记录启动阶段。

第 3 章：建表是最重要的性能设计

3.1 四个不可逃避的问题

建表前必须依次回答：保留明细还是按键更新/聚合？→ 常用过滤条件是什么（排序键）？→ 数据如何按业务范围管理（分区）？→ 一个分区怎样拆成并行 Tablet（分桶）？

3.2 三种表模型

模型	相同 Key 写两次	典型场景	代价/陷阱
DUPLICATE KEY	两行都保留	日志、点击流、不可变明细	不自动去重；更新语义有限
UNIQUE KEY	最终保留最新版本	订单、用户画像、CDC/Upsert	写入维护删除位图/主键索引等；需理解顺序语义
AGGREGATE KEY	按 Value 列聚合函数合并	固定口径报表、预汇总	丢失原始细节；查询口径必须与预聚合相容

【底层】Key 的双重角色

三种模型里的 Key 都影响排序。Duplicate 中它不是唯一约束；Unique 中还定义去重键；Aggregate 中定义聚合粒度。模型创建后不能直接互换，选错通常意味着新建表并回灌。

3.3 三张教学表

```
CREATE DATABASE IF NOT EXISTS doris_lab;
USE doris_lab;

CREATE TABLE logs (
  event_time DATETIME NOT NULL,
  service VARCHAR(32) NOT NULL,
  level VARCHAR(8), message STRING
) DUPLICATE KEY(event_time, service)
PARTITION BY RANGE(event_time) ()
DISTRIBUTED BY HASH(service) BUCKETS 4
PROPERTIES("replication_num"="1",
  "auto_partition_name_prefix"="p");

CREATE TABLE orders (
  order_id BIGINT NOT NULL,
  order_time DATETIME NOT NULL,
  city VARCHAR(32), status VARCHAR(16),
  amount DECIMAL(12,2)
) UNIQUE KEY(order_id)
DISTRIBUTED BY HASH(order_id) BUCKETS 4
PROPERTIES("replication_num"="1");

CREATE TABLE daily_sales (
  dt DATE NOT NULL, city VARCHAR(32) NOT NULL,
  revenue DECIMAL(18,2) SUM,
  order_count BIGINT SUM
) AGGREGATE KEY(dt, city)
DISTRIBUTED BY HASH(city) BUCKETS 4
PROPERTIES("replication_num"="1");
```

【版本核对】自动分区

自动分区的具体 DDL 与属性会随小版本演进。若上述实验语法与你的 3.0.8 包不一致，改用显式 RANGE 分区，并以 SHOW CREATE TABLE 和 3.0.8 对应文档为准。

3.4 用数据证明模型不同

```
INSERT INTO orders VALUES
(1,'2026-08-01 10:00:00','上海','PAID',99.00);
INSERT INTO orders VALUES
(1,'2026-08-01 10:00:00','上海','SHIPPED',99.00);
SELECT * FROM orders WHERE order_id=1;

INSERT INTO daily_sales VALUES ('2026-08-01','上海',100,1);
INSERT INTO daily_sales VALUES ('2026-08-01','上海',250,2);
SELECT * FROM daily_sales;
```

预期：orders 对 order_id=1 最终看到一条最新记录；daily_sales 同一 (dt, city) 的 revenue 与 order_count 分别相加。注意并发写入的“最新”要由 Doris 的序列/事务语义保证，不能把客户端时钟想当然当作真相。

第 4 章：一行数据怎样落到 Tablet

4.1 分区与分桶是两层切分

层	回答的问题	常用依据	主要收益
Partition 分区	这批数据属于哪个业务范围？	日期 RANGE、枚举 LIST	分区裁剪、生命周期、批量删除/替换
Bucket 分桶	分区内部怎样拆成并行小片？	HASH(高频等值/Join 键) 或 RANDOM	并行、负载均衡、减少 Shuffle 的机会
Tablet	最终管理与调度的小分片	一个“分区 × 分桶”产生 Tablet	副本、迁移、修复、扫描调度的基本单位

【底层】路由公式

写入时先根据分区列确定 Partition，再对分桶列计算哈希并取模得到 Bucket/Tablet。查询时，分区谓词可裁剪 Partition；分桶键上的等值条件还可能进一步缩小 Tablet。

4.2 Tablet 数量估算

Tablet 数 $\approx$ 分区数 $\times$ 每分区桶数 $\times$ (每个索引/物化视图)
Replica 数 $\approx$ Tablet 数 $\times$ replication_num

过少：并行度不足、单 Tablet 过大；过多：FE/BE 元数据、文件、Compaction 与调度开销增加。桶数不是“越多越快”。先估算数据量、节点数、查询并发和增长，再压测。

4.3 分桶键选择

- 优先高基数、分布较均匀的列；避免只有少数值的状态列。
- 高频大表 Join 若共享分桶键、桶数和分布，可评估 Colocate，减少网络 Shuffle。
- Unique Key 表通常让分桶列与主键关系清晰，避免更新路由不稳定。
- 不确定时先从合理的小桶数开始，观察单 Tablet 大小与分布再调整。

4.4 副本与多数派

存算一体模式中，Tablet 通常有多个 Replica 分布在不同 BE。写入需要满足副本一致性规则；节点故障后系统可调度修复。学习环境 replication_num=1 仅为降低资源要求，绝不代表生产建议。

4.5 实验

```
SHOW PARTITIONS FROM logs;
SHOW TABLETS FROM orders;
SHOW PROC '/statistic';
EXPLAIN SELECT * FROM orders WHERE order_id=1;
```

- 比较带/不带分区过滤条件的 EXPLAIN 中扫描分区数量。
- 插入大量相同分桶键，观察数据倾斜；再换均匀键对照。
- 计算 365 日分区 $\times$ 32 桶 $\times$ 3 副本的 Replica 数，并解释运维成本。

第 5 章：导入、事务与数据可见性

5.1 如何选择导入方式

来源/需求	优先工具	心智模型
应用少量、低频	JDBC INSERT	简单，但不要逐行高频提交
本地文件/HTTP 高吞吐	Stream Load	客户端推送；同步返回结果
Kafka 持续流	Routine Load	Doris 持续消费任务
Flink/CDC	Flink Doris Connector	上游流处理与 Doris 写入协同
对象存储/HDFS 批量	Broker Load 或 INSERT INTO SELECT	异步批量/外部读取
外部湖仓/数据库	Catalog + INSERT SELECT	先联邦查询，再按需落内表

5.2 Stream Load 完整实验

```
# orders.csv
1,2026-08-01 10:00:00,上海,PAID,99.00
2,2026-08-01 10:05:00,北京,PAID,199.00

curl --location-trusted -u root: \
-H "label:orders_20260801_001" \
-H "column_separator:," \
-H "format:csv" \
-T orders.csv \
http://127.0.0.1:8030/api/doris_lab/orders/_stream_load
```

检查返回 JSON：Status、NumberTotalRows、NumberLoadedRows、NumberFilteredRows、LoadBytes、Message、ErrorURL。成功后再用 SQL 验数，不能只看 HTTP 200。

【概念】Label 与幂等

Label 是一次导入事务的业务标识。网络超时时，客户端可能不知道服务端是否已提交；用稳定、唯一的 Label 查询或重试，可以避免重复导入。幂等表示“重复请求不会重复产生业务效果”。

5.3 写入内部发生了什么

1. FE 接受/协调导入，生成事务并规划数据路由。
2. 数据被解析、转换、校验，按分区和分桶发送到对应 BE。
3. BE 写出新的 Rowset/Segment，并形成副本结果。
4. 满足提交条件后事务进入 COMMITTED；发布成功后进入 VISIBLE，查询才能看到。
5. 失败/超时应以事务与导入状态为准，不凭客户端感受判断。

【底层】原子性与可见性

一批导入要么作为一个事务可见，要么不应让用户看到半成品。COMMITTED 说明提交条件满足；VISIBLE 才代表已发布为可查询版本。分布式系统中“请求返回”和“最终状态”必须区分。

5.4 脏数据治理

- 先定义空值、类型转换、时区、日期格式、超长字符串和枚举规则。
- 设置合理过滤容忍度；ErrorURL 是样本，不是长期数据质量体系。

- 导入后做行数、主键数、金额和、时间边界、空值率等对账。
- 3.0.8 修复了多字符分隔符下 enclose 解析、JSON boolean 到 INT、Stream Load 缺少 ErrorURL 等问题；遇到相似症状先核对准确构建号。

第 6 章：列式存储、Rowset、Segment 与 Compaction

6.1 为什么列式适合分析

行式存储把一行的列放在一起，适合取完整的少量行；列式存储把同一列的值放在一起，分析查询只读取所需列，并能对相似值使用高效编码和压缩。

查询	行式直觉	列式优势
SELECT * WHERE id=?	定位后取整行很自然	可借助主键/短路径优化，但不必然占优
SUM(amount) WHERE dt=...	可能读入无关列	只读 dt、amount；压缩与向量计算更高效
GROUP BY city	逐行解释执行开销大	批量处理一列向量，利于 SIMD 与缓存

6.2 存储层级

Table

- └ Partition (业务范围)
- └ Tablet (分桶后的分片)
- └ Rowset (一次或一批写入形成的数据版本)
- └ Segment (列式数据文件 + 索引)

【概念】索引不是“越多越好”

Zone Map/MinMax、前缀索引、Bloom Filter、Bitmap、倒排索引等用于跳过无关数据，但每种索引都会增加写入、存储或 Compaction 成本。先用查询模式证明收益。

6.3 Compaction 为什么必需

高频写入会产生许多小 Rowset/Segment。查询若逐个打开和合并，元数据、I/O、去重或聚合开销上升。Compaction 在后台把多个版本合并为更少、更大的版本，并按照表模型再次应用去重/聚合规则。

类型/现象	作用	风险信号
Cumulative Compaction	合并较新的增量 Rowset	积压时小文件增多、查询/写入变慢
Base Compaction	把累积版本与基础版本合并	长期不运行导致版本链拉长
Vertical Compaction	按列组进行合并，降低宽表内存压力	配置不当或资源不足会影响后台吞吐
Segment Compaction	导入过程中减少 Segment 数	避免大导入产生过多 Segment 和 -238 错误

【底层】读写放大

写入小批次越碎，后台合并工作越多（写放大）；版本越多，查询要处理的输入越多（读放大）。Group Commit、批量大小与 Compaction 共同平衡实时性和资源。

6.4 Unique Key 的 Merge-on-Write 直觉

同一主键的新版本写入时，系统维护哪些旧行应被屏蔽的信息（如 Delete Bitmap），使读取更容易直接得到最新视图；后台 Compaction 再清理/合并版本。它把一部分合并成本从读侧移向写侧，适合读多、更新分析场景。

6.5 实验与观察

- 连续执行 100 次单行 INSERT，再改为一次批量 INSERT；比较导入、版本与 Compaction 压力。
- 对 Unique 表重复更新同一主键，验证可见结果，并思考物理旧版本何时清理。
- 使用 SHOW TABLET/Compaction 相关管理接口时，先确认 3.0.8 对应命令和权限；不要在生产盲目手工触发。

第 7 章：SQL 如何变成分布式计算

7.1 从文本到结果的流水线

SQL 文本
→ Parser 解析语法
→ Analyzer 绑定表/列、检查类型与权限
→ Logical Plan 逻辑计划
→ Nereids CBO 基于规则、统计信息与代价选计划
→ Physical Plan / PlanFragment
→ FE 调度到多个 BE
→ BE 拆为 Pipeline 与 PipelineTask 执行
→ Exchange 在节点间传输数据
→ 汇总并返回客户端

【概念】CBO

Cost-Based

Optimizer（基于代价的优化器）比较多种等价计划，依赖表/列统计信息估算行数、选择率和成本。统计信息失真时，Join 顺序、分发方式等可能选错。

7.2 MPP 与 Fragment

MPP（Massively Parallel Processing）把一个查询拆成多个可并行阶段。PlanNode 是 Scan、Join、Aggregate 等算子；PlanFragment 是可下发给 BE 的单机执行片段；Exchange/DataSink 连接片段并完成跨 BE 数据传输。

7.3 Join 分发策略

策略	直觉	适用/风险
Broadcast Join	把小表复制到处理大表的各节点	小表很小时省 Shuffle；估算错会放大内存与网络
Shuffle Join	两边按 Join Key 重新分发	大表对大表通用；网络成本高且怕数据倾斜
Colocate Join	相关数据预先同位置，不跨节点搬运	性能好；对分桶键/桶数/副本布局有约束
Bucket Shuffle	只重分发一侧去匹配另一侧桶布局	介于 Broadcast 与全 Shuffle；依赖表设计

7.4 Pipeline 为什么替代 Volcano

传统 Volcano 模型常用算子逐行 next() 调用，函数调用与线程模型开销明显。Doris 3.0 的 Pipeline 把可连续执行的算子组织为管道，使用固定规模线程与可调度 Task，在等待 I/O、网络或下游背压时让出执行权，更充分利用多核并避免“一个实例一个线程”造成线程爆炸。

【概念】 Pipeline Breaker 与 Backpressure
Breaker是需要阻断/重分发的边界，如某些聚合、排序、Exchange；背压表示下游处理不过来时，上游减速，防止内存无限堆积。Local Shuffle 可在 BE 内重新均衡任务输入，缓解局部倾斜。

7.5 向量化与 SIMD

BE 以列块（Block）批量处理数据，一次函数调用操作许多值，更友好地利用 CPU Cache 和 SIMD 指令。它减少虚函数、分支和解释开销，但仍受数据类型转换、分支复杂度、内存带宽与解压成本影响。

7.6 Runtime Filter

Join 的构建侧在运行时生成 Bloom/MinMax/In-list 等过滤器并推送到扫描侧，尽早过滤不可能匹配的行。它不是免费午餐：生成、传输和等待也有成本；3.0 引入更自适应的计算方式。

第 8 章：EXPLAIN、Profile 与系统化调优

8.1 不要从调参数开始

正确顺序是：确认业务目标与基线 → 找到慢 SQL → 检查表模型/分区/分桶/类型/索引 → 用 EXPLAIN 验证计划 → 实际执行并读 Profile → 对症修改 → A/B 验证与回归。

8.2 EXPLAIN 看什么

EXPLAIN VERBOSE SELECT city, SUM(amount) FROM orders WHERE order_time >= '2026-08-01' GROUP BY city;		
线索	健康问题	可能行动
扫描分区/Tablet	是否成功裁剪？	改写谓词；调整分区/分桶
cardinality / rows	估算与实际是否量级一致？	收集统计信息；检查相关列/倾斜
Join order / distribution	小表是否 Broadcast？大表是否过度 Shuffle？	统计信息、Colocate、SQL/模型调整
predicates / pushdown	过滤是否下推到 Scan？	避免不必要函数/隐式类型转换
Fragment / Exchange	网络边界是否过多？	重新审视 Join、聚合与数据布局

8.3 Profile 看什么

- 先找各 Fragment/Operator 的 Active Time 与最大实例，而不是平均值。
- Scan：RowsRead、RowsReturned、BytesRead、I/O、过滤命中；读很多返很少说明裁剪/索引机会。

- Join：Build/Probe 时间、Hash Table 内存、RowsProduced、Runtime Filter。
- Exchange：BytesSent/Received 与等待，判断 Shuffle 和网络。
- 任务最大值远高于中位值通常提示数据倾斜、慢盘或热点。
- PeakMemoryUsage、Spill、等待队列用于区分内存、调度与 I/O 瓶颈。

【底层】关键路径

并行查询的总耗时往往由最慢实例决定。9 个任务 1 秒、1 个任务 30 秒，平均 3.9 秒并不能解释用户等待 30 秒；要找长尾任务的输入量和等待原因。

8.4 常见反模式

反模式	为什么慢	改进
SELECT *	读取无关列、增加网络和序列化	只选需要列
日期列上套函数过滤	可能阻止分区裁剪/索引	改写为范围谓词
小批高频写	Rowset/Segment 与事务开销大	批量、Group Commit
字符串存数值/日期	压缩、比较、统计与隐式转换差	使用准确类型
桶数极大	Tablet 元数据和写入扇出高	按数据量与并行度压测
只加机器不看 Shuffle	网络/倾斜仍是瓶颈	先定位数据运动与长尾

第 9 章：生产运维与故障思维

9.1 四个黄金信号

维度	Doris 中的观察例子
延迟	查询 P50/P95/P99、导入耗时、FE 规划时间、BE 扫描时间
流量	QPS、导入字节/行、扫描字节、网络 Shuffle
错误	失败查询/导入、超时、心跳失败、Replica 不健康、Compaction 错误
饱和度	CPU、内存、磁盘容量/IOPS、网络、线程池、队列、版本数

9.2 日志与证据链

1. 记录准确版本、构建 hash、部署方式、拓扑和故障时间。
2. 保留完整 SQL/DDI、Query ID/Label、错误码和首个异常；不要只截最后一行。
3. 同时对齐 FE、相关 BE、客户端和系统指标的时间线。
4. 最小化复现：相同 Schema、少量样本、稳定步骤、期望与实际。
5. 变更前保存配置和状态；一次只改一个变量；验证后决定回滚或固化。

9.3 高可用基础

生产 FE 通常部署奇数个可选举节点：Master 负责元数据写，Follower 参与选举，Observer

扩展读能力但不参与选举。BE

的数据副本应分布在故障域。多数派协议的目标是故障下保持一致性；“三副本”也不能替代跨集群备份与恢复演练。

【概念】RPO / RTO

RPO允许最多丢多少时间的数据；RTO允许多久恢复服务。备份策略必须由业务目标倒推，并通过恢复演练证明，不是“有文件就算备份”。

9.4 升级 3.0.8 前后清单

- 通读目标版本 Release Notes 的 Behavior Changes、已知问题与兼容限制。
- 备份 FE 元数据和业务数据；验证备份可恢复；保留配置差异。
- 先在预发回放代表性导入、DDL、查询、权限与外部 Catalog。
- 滚动升级时遵守官方版本路径和节点顺序；持续检查 FE/BE Alive、Replica、任务与错误。
- 升级后运行数据对账和性能基线；关注 VARIANT nested 等行为变化。

9.5 安全最小线

- 首次部署立即设置管理员密码；业务按最小权限创建独立账号。
- 限制 8030/9030/9050 等端口暴露范围；跨网段考虑 TLS/网络加密与网关。
- 凭证不写入脚本、日志或 Git；对象存储 AK/SK 使用密钥管理。
- 审计高风险 DDL/DML；删除分区、DROP、恢复和升级必须有变更流程。

第 10 章：从 GitHub 真实问题学习排障

以下案例取自 apache/doris GitHub。Issue 描述只证明“有人报告了该现象”，不自动证明根因或解决方案。回答分为已知事实、推断与安全处理，避免把猜测写成官方结论。

案例 A：3.0.8 Docker FE 在 cgroup v2 启动失败 (#56784)

已知事实：报告环境为 apache/doris:fe-3.0.8、Arch Linux、cgroup v2、容器内 JDK 18；堆栈在 BDB JE 初始化计算内存预算时触发 JDK CgroupV2Subsystem 空指针。Issue 后来因 stale 关闭，页面没有关联修复 PR。

原理：FE 启动 BDB JE 需要确定内存预算，JDK 容器感知代码读取 cgroup 元数据；异常发生在 JDK 平台层，再向上冒泡阻断 FE 元数据环境初始化。

安全处理：先确认主机 cgroup、镜像 digest、容器 JDK 与完整堆栈；在隔离环境验证官方推荐 JDK/更新镜像或已修补构建。不要直接替换生产 FE 的 JDK 后跳过兼容测试；临时 JVM 规避参数也必须以对应 JDK 官方说明为依据。

案例 B：删除 BE storage 后容器等待 FE Master (#58808)

已知事实：报告者在 3 FE/BE 的 3.0.8 集群中删除故障 BE 的 storage 目录，重启后脚本等待 FE Master/BE ready，60 秒退出；Issue 无维护者诊断并因 stale 关闭。

关键判断：删除 storage 是破坏性操作，会移除该 BE 的本地 Tablet/元数据；“日志中 Master IP 正确”不等于 9010/9030/9050 等通信、FE 中的 BE 注册身份、容器环境变量、主机名解析与集群状态都正确。

排查路径：①确认 FE Master 与多数派健康；②从 BE 容器测试 Master 端口；③检查 FE_SERVERS、BE_ADDR/BE_ID 与 DNS/IP 是否变化；④SHOW BACKENDS 看原 BE 条目和心跳错误；⑤核对启动脚本为何 60 秒退出；⑥若按“新 BE”重建，遵循官方下线/添加流程并等待副本修复。不要在唯一副本环境删除 storage。

案例 C：FROM_UNIXTIME 边界与文档理解 (#57576)

已知事实：报告称文档写 3.0.8/3.1.0 起输入范围扩展，但测试大值仍返回 NULL。

回答：先区分“Unix 时间戳数值范围扩大”与“函数是否把小数视为秒/毫秒”。253402271999 秒远超常见 9999 年边界；若意图传毫秒，通常需先换算为秒并核对函数签名与小数精度。用 SELECT VERSION()、SHOW VARIABLES、最小边界值（二分测试）和 3.0.8 对应函数文档确认；若文档与实现不符，再提交包含精确构建号的 issue。

案例 D：JDBC Catalog 与 query() TVF 字段表现不一致 (#60163)

已知事实：3.0.8-rc 构建中，通过 JDBC Catalog 查询 MySQL information_schema.TABLES 的 table_comment 为空，而 query() 透传 SQL 有值。

原理与处理：Catalog 路径通常经过 Doris 的外部表元数据映射、列类型映射和扫描器；query() 透传路径直接让远端执行 SQL，二者并非同一代码路径。先比较 DESCRIBE、远端列类型/字符集、NULL 与空串、列名大小写和执行计划；透传可作临时对照或绕行，但会改变谓词下推、权限与可移植性。

案例 E：把 3.0.8 Release 修复当作排障索引

症状	3.0.8 Release 线索	你要验证
多字符分隔符 + enclose 解析异常	#54581 / #55052	格式、分隔符、准确构建、过滤行样本
JSON boolean 导入 INT 报错	#54397 / #54640	源 JSON 类型、目标列、strict mode、映射
Stream Load 无 ErrorURL	#54115 / #54266	返回 JSON、过滤行数、FE/BE 日志
自连接错误使用 Colocate Join	#54323	EXPLAIN、结果对照、统计信息与 Join 布局
Schema Change 后新 Tablet 空	#53952 / #54064	任务状态、Tablet/Replica、失败清理日志

第 11 章：源码地图 - 从概念找到实现

不要求零基础立刻读懂全部 C++/Java。先学会“从现象找到层级”，再沿关键类、日志关键词与测试用例追踪。源码路径以 apache/doris branch-3.0 为阅读起点，具体文件随提交变化。

目录/模块	语言	对应概念	建议搜索词
fe/fe-core	Java	SQL 解析、Nereids 优化、Catalog、事务、调度、权限	Nereids, Env, OlapTable, GlobalTransactionMgr
be/src/vec	C++	向量化数据结构、函数、算子	Block, Column, VExpr, pipeline
be/src/pipeline	C++	Pipeline、Task、Operator、调度	PipelineTask, Operator, Dependency
be/src/olap	C++	Tablet、Rowset、Segment、Compaction、Delete Bitmap	Tablet, Rowset, Segment, Compaction
gensrc/thrift	Thrift IDL	FE/BE 之间计划与 RPC 数据结构	TPlanFragment, TQueryOptions
regression-test	Groovy/SQL	行为与 bug 的可复现测试	按函数/issue 号搜索

11.1 用一次慢查询读源码

1. 从 Profile 的算子名和日志类名开始，而不是从仓库首页盲读。
2. 在 FE 找物理计划如何生成 Fragment、选择 Join 分发；在 BE 找对应 Operator。
3. 查看接口输入输出、生命周期 open/get_block/close 或 task 调度，而不是一开始钻研算法细节。
4. 找回归测试，理解维护者认为哪些行为必须稳定。
5. 用 git blame / PR / issue 追溯设计背景；区分 branch-3.0 与 master。

11.2 一条 SQL 的源码追踪练习

```
SELECT city, SUM(amount)
FROM orders WHERE order_time >= '2026-08-01'
GROUP BY city;
```

- FE：定位 SQL 解析后的关系节点、过滤、聚合和物理计划。
- Fragment：找 Exchange 是否存在，观察序列化到 Thrift 的计划。
- BE：找 Scan Operator 产出 Block；Filter 对列向量生成选择结果；Aggregate 维护 Hash Table。
- Profile：把 RowsRead、RowsReturned、ActiveTime 对回代码中的计数器。

【底层学习法】三角验证
文档解释设计意图，源码解释当前实现，实验解释实际行为。三者不一致时，先核对版本与构建，再用最小复现判断；不要只信其中一个。

第 12 章：综合项目 - 实时订单分析仓库

12.1 需求

上游 MySQL 订单会更新状态；每秒持续同步。分析侧需要：按天/城市统计销售额；按 order_id 点查；近 30 天趋势；P95 查询 < 2 秒。要求保留可审计的变更来源，并能重放。

12.2 设计答案

决策	选择	理由
明细表	UNIQUE KEY(order_id)	订单按主键更新，查询看到当前状态
排序键	order_id 为主；另以物化视图服务时间聚合	点查优先，同时避免一张表承载所有访问模式
分区	按 order_time 日期 RANGE	近 30 天裁剪与历史生命周期
分桶	HASH(order_id)，压测确定桶数	均匀分布和点查路由；注意热点业务 ID
导入	Flink CDC / Connector；Label 与 checkpoint 协同	持续增量和故障恢复
汇总	异步/同步物化视图或 Aggregate 汇总表	减少仪表盘重复扫描

12.3 验收实验

- 正确性：插入、更新、删除/撤单；按主键和汇总口径对账。
- 故障：导入中断、BE 重启、网络超时；验证幂等与恢复。
- 性能：10/100/1000 并发下记录 P50/P95/P99、CPU、内存、扫描字节和 Shuffle。
- 数据量：从百万到亿级，观察 Tablet 大小、Compaction、统计信息和计划变化。
- 运维：模拟磁盘高水位与单 BE 故障；验证告警、Replica 修复与容量预案。

12.4 你必须能口述的答案

- 一条更新如何从 Connector 变成可查询的新版本？
- 为什么 order_time 分区但 order_id 分桶？
- 点查为何在 3.0.8 可减少 Fragment？
- 同一个 SQL 突然变慢，如何区分统计信息、倾斜、I/O 和 Compaction？
- 为什么单副本测试成功不能证明生产可靠？

附录 A：基础概念注解表

术语	注解
Catalog	数据源命名空间；internal 指 Doris 内表，外部 Catalog 可连接 Hive/JDBC/Iceberg 等。
Database	一组表的逻辑容器，用于业务隔离与权限管理。
Schema	表的列、类型、键、分区、分桶及属性定义。
DDL / DML / DQL	定义结构 / 修改数据 / 查询数据的 SQL 类别。
FE / BE	前端控制与规划节点 / 后端存储与执行节点。
Partition	第一层数据切分，常按时间；便于裁剪和生命周期管理。
Bucket	分区内部第二层切分，决定 Tablet 和并行度。
Tablet / Replica	最小数据管理分片 / Tablet 的物理副本。
Rowset / Segment	一次写入形成的数据版本集合 / 其中的列式文件。
Compaction	后台合并多个版本，降低读放大并应用模型规则。
MPP	把查询拆到多节点大规模并行执行。
CBO	依据统计信息估算代价并选择物理计划。
PlanNode / Fragment	执行算子 / 可下发到 BE 的计划片段。
Exchange / Shuffle	片段间数据通道 / 按键等规则跨节点重分布数据。
Pipeline / Task	BE 内算子管道 / 可调度的执行任务。
Vectorization / SIMD	批量列处理 / 一条 CPU 指令并行处理多个值。
Runtime Filter	查询运行中由 Join 等生成并推到扫描端的动态过滤器。
Materialized View	预计算并存储的查询结果，优化器可用于改写加速。
Statistics	表/列数据量、基数、分布等，供 CBO 估算。
Profile	实际执行的时间、行数、字节、内存与等待等度量。
Label	导入任务的唯一业务标识，用于状态查询和幂等。
COMMITTED / VISIBLE	事务已满足提交 / 已发布为查询可见版本。
CDC / Upsert	捕获上游变更 / 按键插入或覆盖更新。
RPO / RTO	可容忍数据损失窗口 / 可容忍恢复时长。

附录 B：面试与自测题（含简答）

B.1 基础题

• Q：FE 宕机是否意味着数据文件丢失？

A：通常数据在 BE；但 FE 元数据/选举不可用会影响访问与管理。必须分别判断控制面与数据面。

• Q：Partition 和 Bucket 有何区别？

A：Partition 按业务范围切分并支持裁剪/生命周期；Bucket 在分区内产生 Tablet，影响并行和数据分布。

• Q：Duplicate Key 的 Key 唯一吗？

A：不唯一，主要是排序键。

• Q：为何小批写入多会拖慢？

A：事务、Rowset/Segment 和后续 Compaction 固定开销被放大。

B.2 原理题

• Q：Broadcast Join 为什么可能 OOM？

A：构建侧被复制到多个执行实例；统计低估或并发高会把内存放大。

• Q：Pipeline 怎样减少线程爆炸？

A：把执行拆成可调度 Task，由受控线程池运行；阻塞时让出而非长期占一个线程。

• Q：Unique Key 为什么需要 Delete Bitmap/合并？

A：同键旧物理行可能仍在旧 Segment，需要标记不可见并在 Compaction 中清理。

• Q：为什么 Profile 要看最大实例？

A：并行查询由关键路径/长尾决定，平均值会掩盖倾斜。

B.3 排障题

• Q：Stream Load HTTP 成功但数据少了怎么办？

A：检查返回 JSON 的 Total/Loaded/Filtered、ErrorURL、strict mode、列映射，再按 Label 查询状态并做业务对账。

• Q：查询突然变慢的最短证据链？

A：锁定 Query ID 与基线 → EXPLAIN 计划/估算 → Profile 最大算子与实例 → 系统 CPU/内存/I/O/网络 → 近期 Schema/统计/数据分布/Compaction 变化。

• Q：能直接删除故障 BE 的 storage 吗？

A：不能作为默认操作。先确认副本与正式下线/替换流程；删除会永久丢失该节点本地数据。

附录 C：资料、版本与引用

访问日期：2026-08-13。优先使用固定 3.0.8 Release 与 3.x 文档；官网 “3.x” 可能持续更新，涉及命令/属性时请切换到与安装包一致的版本页面。

资料	链接 / 用途
3.0.8 Release Notes	https://doris.apache.org/releases/v3.0/release-3.0.8/ – 行为变化、改进与修复
3.x Quick Start	https://doris.apache.org/docs/3.x/gettingStarted/quick-start/ – Docker 与本地部署
What is Doris	https://doris.apache.org/docs/3.x/gettingStarted/what-is-apache-doris/ – FE/BE 与两种架构
Data Distribution	https://doris.apache.org/docs/3.x/table-design/data-partitioning/data-distribution/ – 分区、分桶、Tablet
Table Design	https://doris.apache.org/docs/3.x/table-design/overview/ – 建模路线
Loading Overview	https://doris.apache.org/docs/3.x/data-operate/import/load-manual/ – 导入选择
Parallel Execution	https://doris.apache.org/docs/3.x/query-acceleration/optimization-technology-principle/pipeline-execution-engine/ – Pipeline
Analysis Tools	https://doris.apache.org/docs/3.x/query-acceleration/performance-tuning-overview/analysis-tools/ – EXPLAIN/Profile
Compaction	https://doris.apache.org/docs/3.x/admin-manual/trouble-shooting/compaction/ – 合并机制与排障
源码 branch-3.0	https://github.com/apache/doris/tree/branch-3.0 – 实现与回归测试
GitHub #56784	https://github.com/apache/doris/issues/56784 – cgroup v2/JDK 启动报告
GitHub #58808	https://github.com/apache/doris/issues/58808 – BE storage 删除后启动报告
GitHub #57576	https://github.com/apache/doris/issues/57576 – FROM_UNIXTIME 边界报告
GitHub #60163	https://github.com/apache/doris/issues/60163 – Catalog 与 query() 差异报告

最后提醒

GitHub issue 可能缺少维护者结论、复现条件或后续修复。生产操作必须回到对应版本官方文档、Release Notes、已合并 PR/commit 与自己的隔离验证。

学习不是记住 Doris 的所有参数，而是能从数据语义、物理布局和执行证据推导答案。